OCR H446 / PAPER 1 / UNIT 1.4

Data types, data structures and Boolean algebra

CHAPTER CONTENTS

Select a topic to jump to its original worked pages.

1.4.1(a) / Primitive data types	5
1.4.1(b,e,f) / Unsigned binary and hexadecimal	9
1.4.1(c) / Negative binary values	13
1.4.1(d) / Binary addition and subtraction	17
1.4.1(g) / Normalised binary floating point	21
1.4.1(h) / Signed floating-point arithmetic	25
1.4.1(i) / Bitwise logic and shifts	29
1.4.1(j) / ASCII and Unicode	33
1.4.2(a) / Arrays, records, lists and tuples	37
1.4.2(b,c) / Stacks: push, pop and peek	41
1.4.2(b,c) / Queues and circular buffers	45
1.4.2(b,c) / Linked-list operations	49
1.4.2(b,c) / Trees and binary search trees	53
1.4.2(c) / Tree traversal and deletion	57
1.4.2(b,c) / Graphs and adjacency structures	61
1.4.2(b,c) / Hash tables and collisions	65
1.4.3(a,d) / Boolean problems and truth tables	69

Full-paper guide retains further synoptic practice and reference appendices.

OCR H446 / PAPER 1 / UNIT 1.4

Data types, data structures and Boolean algebra

CHAPTER CONTENTS

Select a topic to jump to its original worked pages.

1.4.3(d) / Logic gates and combined circuits	73
1.4.3(c) / Boolean laws and De Morgan	77
1.4.3(b) / Three-variable Karnaugh maps	81
1.4.3(b) / Four-variable K-maps	85
1.4.3(e) / Half adders	89
1.4.3(e) / Full adders	93
1.4.3(e) / D-type flip-flops	97

H446/01 / REVISION

Understand it. Model it. Try it. Check it.

Use this chapter to understand the theory, follow worked models and practise independently.

THREE SPECIFICATION FAMILIES

1.4.1 DATA TYPES + REPRESENTATION	Unsigned and signed binary, floating point, masks and Unicode
1.4.2 DATA STRUCTURES	Arrays, records, linked structures, trees, graphs and hashing
1.4.3 BOOLEAN ALGEBRA	Truth tables, K-maps, circuits, adders and flip-flops

FOUR PASSES FOR EVERY TOPIC

01 UNDERSTAND
Build a precise conceptual picture.

02 MODEL
Follow an explicit trace or diagram.

03 TRY
Try original exam-style questions.

04 CHECK
Self-mark with separate mark points.

revision questions, page maps and bookmarks are not changed by this PDF.

Show binary working, data-structure operations and every step of Boolean simplification.

1.4 / SPECIFICATION CHECKLIST

Every 1.4 topic has a working model

Select a linked row to jump to its Understand page. The following pages contain a model, exam-style practice and indicative answer points.

SPEC	LEARNING CYCLE	PAGES
1.4.1(a)	Primitive data types	3-6
1.4.1(b,e,f)	Unsigned binary and hexadecimal	7-10
1.4.1(c)	Negative binary values	11-14
1.4.1(d)	Binary addition and subtraction	15-18
1.4.1(g)	Normalised binary floating point	19-22
1.4.1(h)	Signed floating-point arithmetic	23-26
1.4.1(i)	Bitwise logic and shifts	27-30
1.4.1(j)	ASCII and Unicode	31-34
1.4.2(a)	Arrays, records, lists and tuples	35-38
1.4.2(b,c)	Stacks: push, pop and peek	39-42
1.4.2(b,c)	Queues and circular buffers	43-46
1.4.2(b,c)	Linked-list operations	47-50
1.4.2(b,c)	Trees and binary search trees	51-54
1.4.2(c)	Tree traversal and deletion	55-58
1.4.2(b,c)	Graphs and adjacency structures	59-62
1.4.2(b,c)	Hash tables and collisions	63-66
1.4.3(a,d)	Boolean problems and truth tables	67-70
1.4.3(d)	Logic gates and combined circuits	71-74
1.4.3(c)	Boolean laws and De Morgan	75-78
1.4.3(b)	Three-variable Karnaugh maps	79-82
1.4.3(b)	Four-variable K-maps	83-86
1.4.3(e)	Half adders	87-90
1.4.3(e)	Full adders	91-94
1.4.3(e)	D-type flip-flops	95-98

SPECIFICATION BOUNDARY

1.4.1(a-j), 1.4.2(a-c), and 1.4.3(a-e) from OCR H446 specification v2.7. Models state their representation/indexing conventions.

Source: OCR H446 A Level Computer Science specification v2.7 (2024), section 1.4; compared against current RHSC guide section map.

1.4.1(a) / BUILD THE MENTAL MODEL

Choose a representation that fits the value

An appropriate data type constrains the values and operations of a variable. Data types are not all stored as simple whole numbers.

THREE PIECES OF THE IDEA

INTEGER

Whole-number values, positive or negative, subject to a chosen range.

REAL / FLOAT

Approximates values with fractional parts using a limited number of bits.

OTHER PRIMITIVES

Boolean holds true/false; character is one symbol; string holds a sequence of characters.

MODEL / FOLLOW THE FLOW

Problem value → **Select data type** → **Check valid range** → **Choose operations**

Number of students: integer. Temperature 18.5: real. Permission granted: Boolean. Initial: character. Full surname: string.

EXAM-READY STATEMENT

Selection depends on the meaning and valid operations of the value, not merely whether quotation marks appear in pseudocode.

WORDS THAT EARN MARKS

Integer Whole number

Real Approximate fraction

Boolean True or false

Character Single symbol

1.4.1(a) / STEP-BY-STEP WORKED EXAMPLE

Declare variables for a library loan

Follow each stage in order. Explain both the change and why it happens before you try the exam questions.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
1. Count	copies = 12 / integer	You cannot have a fraction of a physical copy.
2. Price	fine = 1.25 / real	It includes a fractional component.
3. Flag	overdue = TRUE / Boolean	Only two logical states are needed.
4. Code	initial = "K" / character	Exactly one character is stored.
5. Text	title = "Data Structures" / string	A string is an ordered sequence of characters.

TRANSFER IT TO AN EXAM ANSWER

Selection depends on the meaning and valid operations of the value, not merely whether quotation marks appear in pseudocode.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check intermediate values and key relationships, not just the final result.

1.4.1(a) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Original teacher-written practice questions (not OCR past-paper questions). Work in black pen; use the next page to check.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 5 MARKS

Choose a type for pupilCount, temperature, hasPaid, surname and firstInitial. Justify two choices.

QUESTION 2 | 4 MARKS

Explain why a real/floating-point value may be inappropriate for comparing financial totals exactly.

1.4.1(a) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 5 MARKS

- Integer, real, Boolean, string and character respectively.
- Count uses whole numbers; hasPaid has two possible states.

Q2 / 4 MARKS

- Many decimal fractions cannot be represented exactly in binary floating point.
- Small representation/rounding errors can occur.
- A suitable fixed-point/integer-minor-unit representation can preserve exact pence.

COMMON MISCONCEPTION

A floating-point value can be approximate even when it appears with only one or two decimal places.

Green pen: Always attach the example value to the reason for selecting the type.

1.4.1(b,e,f) / BUILD THE MENTAL MODEL

Move between bases without losing place value

Binary uses powers of 2; hexadecimal uses powers of 16. One hex digit maps exactly to four binary bits.

THREE PIECES OF THE IDEA

BINARY

Read bits from right to left as 1, 2, 4, 8, 16, 32 and so on.

HEXADECIMAL

Digits 0-9 and A-F represent values 0-15.

NIBBLE

Group a binary word from the right into four-bit groups, padding on the left if needed.

MODEL / FOLLOW THE FLOW

Denary 173 → **Binary 10101101** → **Nibbles 1010 1101** → **Hex AD**

173 = 128 + 32 + 8 + 4 + 1; binary 10101101; 1010 is A and 1101 is D.

EXAM-READY STATEMENT

Write out place values for denary conversions. Use exact nibble mapping for binary/hex conversions.

WORDS THAT EARN MARKS

Base Radix

Bit Binary digit

Nibble 4 bits

Place value Power of the base

1.4.1(b,e,f) / STEP-BY-STEP WORKED EXAMPLE

Convert 0x3C to binary and denary

Follow each stage in order. Explain both the change and why it happens before you try the exam questions.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
1. Hex digits	3 C	C is 12 in denary.
2. Nibble 1	3 becomes 0011	Each hex digit is exactly four bits.
3. Nibble 2	C becomes 1100	12 = 8 + 4.
4. Join	00111100	Maintain eight positions.
5. Denary	3 x 16 + 12 = 60	Also 32 + 16 + 8 + 4 = 60.

TRANSFER IT TO AN EXAM ANSWER

Write out place values for denary conversions. Use exact nibble mapping for binary/hex conversions.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check intermediate values and key relationships, not just the final result.

1.4.1(b,e,f) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Original teacher-written practice questions (not OCR past-paper questions). Work in black pen; use the next page to check.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 4 MARKS

Convert binary 11010110 to hexadecimal and denary. Show working.

QUESTION 2 | 4 MARKS

Convert denary 157 into 8-bit binary and hexadecimal. Show the place values.

1.4.1(b,e,f) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 4 MARKS

- 1101 0110 = D6.
- 128 + 64 + 16 + 4 + 2 = 214.

Q2 / 4 MARKS

- 157 = 128 + 16 + 8 + 4 + 1.
- 10011101 = 9D hexadecimal.

COMMON MISCONCEPTION

Do not map individual bits to hex digits; each hex digit replaces exactly four bits.

Green pen: Write the nibble grouping line explicitly before converting.

1.4.1(c) / BUILD THE MENTAL MODEL

Sign-and-magnitude is not two’s complement

Two different signed representations can give different bit patterns for the same negative value. State the width of the word.

THREE PIECES OF THE IDEA

SIGN / MAGNITUDE

Most significant bit gives sign; remaining bits are the magnitude.

TWO'S COMPLEMENT

Invert an 8-bit positive representation and add 1; one zero pattern exists.

RANGE

8-bit two's complement: -128 to +127. Sign/magnitude: -127 to +127 with two zeros.

MODEL / FOLLOW THE FLOW

+13 = 00001101 → **Invert 11110010** → **Add 1 = 11110011** → **Result = -13**

In 8 bits, sign/magnitude -13 is 10001101, but two's-complement -13 is 11110011.

EXAM-READY STATEMENT

Specify word size, representation and the conversion method; two's complement is not simply setting the top bit.

WORDS THAT EARN MARKS

- **MSB** Sign position
- **Magnitude** Absolute value
- **Invert** Flip all bits
- **Overflow** Out of range

1.4.1(c) / STEP-BY-STEP WORKED EXAMPLE

Represent -23 in both 8-bit conventions

Follow each stage in order. Explain both the change and why it happens before you try the exam questions.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
1. Positive	+23 = 00010111	16 + 4 + 2 + 1.
2. Sign/mag	10010111	Set sign bit; retain 7-bit magnitude.
3. Invert	11101000	Flip all eight bits of +23.
4. Add one	11101001	This encodes -23 in two's complement.
5. Check	256 - 233 = 23	Unsigned value 233 represents -23 in 8 bits.

TRANSFER IT TO AN EXAM ANSWER

Specify word size, representation and the conversion method; two's complement is not simply setting the top bit.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check intermediate values and key relationships, not just the final result.

1.4.1(c) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Original teacher-written practice questions (not OCR past-paper questions). Work in black pen; use the next page to check.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 4 MARKS

Write -42 in 8-bit sign-and-magnitude and 8-bit two's complement. Show the invert/add-one step.

QUESTION 2 | 4 MARKS

State the range of 8-bit two's-complement values and explain why there is only one zero.

1.4.1(c) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 4 MARKS

- +42 = 00101010. Sign/magnitude = 10101010.
- Invert -> 11010101; add 1 -> 11010110.

Q2 / 4 MARKS

- -128 through +127 inclusive.
- Zero is 00000000; inversion and adding one wraps zero back to the same pattern.
- There is no separate negative-zero representation.

COMMON MISCONCEPTION

Setting the sign bit in a two's-complement number generally changes its magnitude.

Green pen: Check the decoded value using negative MSB weight or invert/add-one.

1.4.1(d) / BUILD THE MENTAL MODEL

Use fixed width and detect overflow

Binary arithmetic uses carries. In fixed-width two's complement, an out-of-range result wraps; overflow must be recognised.

THREE PIECES OF THE IDEA

ADDITION

0+0=0, 0+1=1, 1+1=10, 1+1+1=11 in binary.

SUBTRACTION

A - B can be performed as A plus the two's complement of B.

OVERFLOW

Two same-sign operands giving the opposite-sign result indicates signed overflow.

MODEL / FOLLOW THE FLOW

13 = 00001101 → **-5 = 11111011** → **Add = 00001000** → **Answer +8**

The ninth carry bit is discarded in an eight-bit register; 00001000 equals +8.

EXAM-READY STATEMENT

A carry out is not the same test as signed overflow. Check operand signs and result sign.

WORDS THAT EARN MARKS

Carry	Column overflow	**Width**	Number of bits
Wrap	Discard excess bit	**Signed**	Two's complement

1.4.1(d) / STEP-BY-STEP WORKED EXAMPLE

Calculate 19 - 7 as 8-bit addition

Follow each stage in order. Explain both the change and why it happens before you try the exam questions.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
1. A	19 = 00010011	An eight-bit positive operand.
2. +7	00000111	Write out the subtrahend.
3. Negate	11111001	Invert 00000111 -> 11111000; add 1.
4. Add	00010011 + 11111001 = 1 00001100	Discard the ninth carry in fixed width.
5. Decode	00001100 = +12	19 - 7 = 12; no signed overflow.

TRANSFER IT TO AN EXAM ANSWER

A carry out is not the same test as signed overflow. Check operand signs and result sign.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check intermediate values and key relationships, not just the final result.

1.4.1(d) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Original teacher-written practice questions (not OCR past-paper questions). Work in black pen; use the next page to check.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 5 MARKS

Use eight-bit two's complement to calculate 14 - 9, displaying the bit patterns.

QUESTION 2 | 4 MARKS

Explain why 01111111 + 00000001 causes signed overflow in eight bits.

1.4.1(d) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 5 MARKS

- 14 = 00001110; +9 = 00001001; -9 = 11110111.
- 00001110 + 11110111 = 1 00000101 -> 00000101 = +5.

Q2 / 4 MARKS

- 01111111 is +127 and 00000001 is +1.
- Expected +128 is outside [-128,127].
- Stored result 10000000 decodes as -128: two positives produce a negative.

COMMON MISCONCEPTION

A carry from the most significant bit alone does not determine signed overflow.

Green pen: Check expected mathematical result against range and signs.

1.4.1(g) / BUILD THE MENTAL MODEL

Treat mantissa as a signed binary fraction

For these worked examples the format is an 8-bit two's-complement fractional mantissa followed by a 4-bit signed exponent.

THREE PIECES OF THE IDEA

VALUE

Number = mantissa x 2 raised to exponent; the binary point sits immediately after the sign bit.

NORMALISE

Positive mantissa begins 01; negative mantissa begins 10 (except zero).

PRECISION

Mantissa length limits precision; exponent range limits scale. Truncation/rounding can lose bits.

WORKED VISUAL / NOT TO SCALE

SIGN + FRACTION (8 BITS)

0	1	1	0	1	0	0	0

EXPONENT 4 BITS

0	0	1	1

0.1101000 x 2^3 = 0.8125 x 8 = 6.5

EXAM-READY STATEMENT

Before any calculation, write the mantissa/exponent widths and the location of the binary point.

WORDS THAT EARN MARKS

Mantissa Significant bits

Exponent Power of two

Normalised Leading 01 or 10

Precision Representable detail

1.4.1(g) / STEP-BY-STEP WORKED EXAMPLE

Encode +6.5 and -6.5 in the 8+4 scheme

Follow each stage in order. Explain both the change and why it happens before you try the exam questions.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
1. Convert	6.5 = 110.1 binary	110 base 2 = 6; .1 base 2 = 0.5.
2. Normalise	+0.1101 x 2^3	Positive starts 01 when the sign bit is included.
3. Encode +	01101000 0011	8-bit mantissa, 4-bit exponent +3.
4. Encode -	10011000 0011	Two's-complement mantissa of -0.8125.
5. Check	-104/128 x 8 = -6.5	Negative mantissa begins 10.

TRANSFER IT TO AN EXAM ANSWER

Before any calculation, write the mantissa/exponent widths and the location of the binary point.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check intermediate values and key relationships, not just the final result.

1.4.1(g) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Original teacher-written practice questions (not OCR past-paper questions). Work in black pen; use the next page to check.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 5 MARKS

In an 8-bit signed fractional mantissa and 4-bit signed exponent format, represent +5 using a normalised mantissa.

QUESTION 2 | 4 MARKS

Explain the difference between the precision and range of floating-point representations.

1.4.1(g) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 5 MARKS

- 5 = 101 binary = 0.101 x 2^3.
- Mantissa 01010000, exponent 0011; value 0.625 x 8 = 5.

Q2 / 4 MARKS

- More mantissa bits provide more significant precision.
- More exponent bits enable a wider range of magnitudes.
- Finite bits still cause rounding/representation error.

COMMON MISCONCEPTION

Do not interpret the mantissa as an ordinary unsigned integer. It is a signed binary fraction.

Green pen: State the convention and calculate the mantissa decimal fraction to check.

1.4.1(h) / BUILD THE MENTAL MODEL

Align exponent, operate on mantissas, normalise

To add or subtract binary floating-point values, align exponents first, then operate on signed mantissas and normalise.

THREE PIECES OF THE IDEA

ALIGN

Shift the mantissa of the smaller exponent right until exponents match.

OPERATE

Add signed mantissas; for subtraction, add a negated mantissa.

RENORMALISE

Shift mantissa and change exponent in opposite directions; lost low bits can reduce precision.

MODEL / FOLLOW THE FLOW

2 = .5 x 2^2 → **-1 = -.5 x 2^1** → **Align -1 to -.25 x 2^2** → **Result .25 x 2^2 = 1**

Intermediate 8-bit mantissa 00100000 is not normalised; shift left to 01000000 and reduce exponent from 2 to 1.

EXAM-READY STATEMENT

Keep mantissa and exponent changes together. When adding positive values, reserve an extra bit if a mantissa sum overflows.

WORDS THAT EARN MARKS

- **Align** Same exponent
- **Shift** Move binary point
- **Renormalise** Restore 01 or 10
- **Rounding** Precision loss

1.4.1(h) / STEP-BY-STEP WORKED EXAMPLE

Use 8+4 signed-fraction format to compute 2 - 1

Follow each stage in order. Explain both the change and why it happens before you try the exam questions.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
1. Encode +2	01000000 0010	0.5 x 2^2 = 2.
2. Encode -1	11000000 0001	-0.5 x 2^1 = -1.
3. Align -1	11100000 0010	-0.25 x 2^2 = -1.
4. Add	00100000 0010	0.25 x 2^2 = 1; mantissa starts 00.
5. Normalise	01000000 0001	Shift left one and decrease exponent to 1.

TRANSFER IT TO AN EXAM ANSWER

Keep mantissa and exponent changes together. When adding positive values, reserve an extra bit if a mantissa sum overflows.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check intermediate values and key relationships, not just the final result.

1.4.1(h) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Original teacher-written practice questions (not OCR past-paper questions). Work in black pen; use the next page to check.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 5 MARKS

Show the aligned mantissas and normalised result for +3 plus -1 using the stated 8+4 signed-fraction format.

QUESTION 2 | 4 MARKS

Explain why aligning exponents before addition can reduce precision.

1.4.1(h) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 5 MARKS

- +3: 01100000 0010 (0.75 x 4).
- -1: 11000000 0001; align to 11100000 0010 (-0.25 x 4).
- Add = 01000000 0010: 0.5 x 4 = +2, normalised.

Q2 / 4 MARKS

- Right-shifting the smaller-exponent mantissa discards low-order bits if width is fixed.
- Once discarded, those bits cannot be recovered by normalisation.

COMMON MISCONCEPTION

A binary string that starts 00 or 11 is not a normalised nonzero mantissa in this convention.

Green pen: Write each exponent beside its mantissa; verify the final denary value.

1.4.1(i) / BUILD THE MENTAL MODEL

A mask edits selected bit positions

Bitwise AND, OR and XOR operate independently on corresponding bits. Logical shifts move bits and fill newly empty positions with zeros.

THREE PIECES OF THE IDEA

AND MASK

AND with 1 preserves a bit; AND with 0 clears it.

OR / XOR

OR with 1 sets a bit. XOR with 1 toggles a bit.

SHIFTS

Logical left/right shifts move positions; bits shifted off the fixed width are lost.

MODEL / FOLLOW THE FLOW

Value 10110110 → **AND mask 00001111** → **Output 00000110** → **Keep low nibble**

A mask must be aligned to the same bit positions as the value being modified.

EXAM-READY STATEMENT

For fixed-width logical shifts, label direction, inserted zeros, discarded bits and chosen bit width.

WORDS THAT EARN MARKS

- **AND** Both bits 1
- **OR** At least one 1
- **XOR** Exactly one 1
- **Mask** Bit selector

1.4.1(i) / STEP-BY-STEP WORKED EXAMPLE

Modify the 8-bit value 10110110

Follow each stage in order. Explain both the change and why it happens before you try the exam questions.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
1. Clear upper	AND 00001111 -> 00000110	Zeros clear the high nibble.
2. Set bit 0	OR 00000001 -> 10110111	One forces the low bit on.
3. Toggle all	XOR 11111111 -> 01001001	XOR with one flips each bit.
4. Shift right	10110110 logical >> 2 = 00101101	Insert zeros at the left.
5. Check width	Discard bits shifted beyond 8 places	Do not extend the register indefinitely.

TRANSFER IT TO AN EXAM ANSWER

For fixed-width logical shifts, label direction, inserted zeros, discarded bits and chosen bit width.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check intermediate values and key relationships, not just the final result.

1.4.1(i) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Original teacher-written practice questions (not OCR past-paper questions). Work in black pen; use the next page to check.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 4 MARKS

Starting at 11010101, apply AND 11110000 then OR 00000011. Show both intermediate words.

QUESTION 2 | 4 MARKS

Explain how XOR can toggle a flag bit without changing other bits. Give an 8-bit example.

1.4.1(i) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 4 MARKS

- 11010101 AND 11110000 = 11010000.
- 11010000 OR 00000011 = 11010011.

Q2 / 4 MARKS

- XOR with 1 toggles; XOR with 0 leaves unchanged.
- For 10110000, XOR 00000100 gives 10110100 (bit 2 changes).

COMMON MISCONCEPTION

A logical right shift is not always division by 2 for signed two's-complement values; signed/arithmetic shifts need separate treatment.

Green pen: Number the bit positions from the right before selecting a mask.

1.4.1(j) / BUILD THE MENTAL MODEL

Character and encoding are different concepts

A character set assigns identities to symbols. An encoding determines the bytes used to store those identities.

THREE PIECES OF THE IDEA

ASCII

Original ASCII uses seven bits and represents 128 character codes.

UNICODE

Assigns code points to characters across many writing systems and symbols.

UTF-8

A variable-length Unicode encoding; ASCII-range characters use one byte, others may require more.

MODEL / FOLLOW THE FLOW

Letter A → **Code point U+0041** → **UTF-8 byte 41** → **Decode to A**

Unicode code point U+20AC (euro sign) is stored in UTF-8 as three bytes E2 82 AC.

EXAM-READY STATEMENT

Do not equate Unicode with a fixed 16-bit or 32-bit encoding; Unicode characters can be encoded using UTF-8, UTF-16 or UTF-32.

WORDS THAT EARN MARKS

- **Code point** Character identifier
- **ASCII** 7-bit set
- **UTF-8** Variable-length bytes
- **Glyph** Rendered symbol

1.4.1(j) / STEP-BY-STEP WORKED EXAMPLE

Compare two characters sent in UTF-8

Follow each stage in order. Explain both the change and why it happens before you try the exam questions.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
1. A	U+0041	An ASCII-range Unicode code point.
2. A bytes	41 hex	One UTF-8 byte.
3. Euro	U+20AC	Outside original 7-bit ASCII.
4. Euro bytes	E2 82 AC	Three UTF-8 bytes.
5. Interpret	One character need not equal one byte	Storage size depends on encoding, not character count alone.

TRANSFER IT TO AN EXAM ANSWER

Do not equate Unicode with a fixed 16-bit or 32-bit encoding; Unicode characters can be encoded using UTF-8, UTF-16 or UTF-32.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check intermediate values and key relationships, not just the final result.

1.4.1(j) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Original teacher-written practice questions (not OCR past-paper questions). Work in black pen; use the next page to check.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 4 MARKS

Explain why the text file containing “A” and “€” may need different numbers of bytes per character.

QUESTION 2 | 4 MARKS

Distinguish a Unicode code point from UTF-8, giving an example.

1.4.1(j) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 4 MARKS

- A uses one byte in UTF-8 because it is in the ASCII range.
- Euro sign U+20AC requires three UTF-8 bytes (E2 82 AC).
- UTF-8 is variable-length.

Q2 / 4 MARKS

- Code point identifies a character, e.g. U+0041.
- UTF-8 encodes that identifier as bytes, e.g. 0x41.
- Same Unicode character may have another byte sequence in UTF-16.

COMMON MISCONCEPTION

Unicode is not the name of a single fixed-width bit pattern.

Green pen: Always name the encoding when giving a byte count.

1.4.2(a) / BUILD THE MENTAL MODEL

Choose a structure by shape and behaviour

These structures differ in whether they have fixed positions, named fields, changing length or immutable groups.

THREE PIECES OF THE IDEA

ARRAY

Indexed elements of a common type; OCR includes one-, two- and three-dimensional arrays.

RECORD

Named fields of potentially different types; e.g. Student.Name and Student.Age.

LIST / TUPLE

A list is an ordered, commonly mutable collection; a tuple is an ordered fixed group, often immutable.

MODEL / FOLLOW THE FLOW

Sensor[day] → **Grid[row,col]** → **Voxel[x,y,z]** → **Record fields**

A 3D array index has three coordinates. A record is better when each item holds different named attributes.

EXAM-READY STATEMENT

Array indices locate elements; record field names identify attributes. A 3D array is not the same as three separate arrays.

WORDS THAT EARN MARKS

- **Index** Position
- **Dimension** Independent index
- **Field** Named attribute
- **Tuple** Fixed ordered group

1.4.2(a) / STEP-BY-STEP WORKED EXAMPLE

Represent six measurements and one student

Follow each stage in order. Explain both the change and why it happens before you try the exam questions.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
1. Array	temp[0..5]	A 1D sequence of six same-type readings.
2. Matrix	seat[1..3,1..4]	A 2D grid with 3 rows and 4 columns.
3. Volume	pixel[0..1,0..2,0..3]	A 3D array with 2 x 3 x 4 = 24 elements.
4. Record	student = (name, year, present)	Fields may have string, integer, Boolean types.
5. List	tasks.add(newTask)	The number of tasks can grow dynamically.

TRANSFER IT TO AN EXAM ANSWER

Array indices locate elements; record field names identify attributes. A 3D array is not the same as three separate arrays.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check intermediate values and key relationships, not just the final result.

1.4.2(a) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Original teacher-written practice questions (not OCR past-paper questions). Work in black pen; use the next page to check.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 5 MARKS

A game stores blocks in a 4 x 3 x 2 region. Choose a data structure, state its capacity and give an access expression.

QUESTION 2 | 4 MARKS

Compare a record with an array for storing a pupil's name, year and attendance flag.

1.4.2(a) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 5 MARKS

- A three-dimensional array is suitable.
- Capacity = 4 x 3 x 2 = 24 elements.
- For zero-based indices, e.g. block[3,2,1] is a valid access.

Q2 / 4 MARKS

- Record has named fields and allows different data types.
- A conventional array has indexed elements of a single type.
- Use Student.Name, Student.Year, Student.Present.

COMMON MISCONCEPTION

Do not infer zero-based indexing unless a question defines it; state your assumption.

Green pen: Draw the dimensions and label the index order before calculating capacity.

1.4.2(b,c) / BUILD THE MENTAL MODEL

Last in, first out means top changes first

A stack permits insertion and removal at the top. A bounded array stack must detect full and empty states.

THREE PIECES OF THE IDEA

PUSH

Check capacity; increment top pointer and store a new item.

POP

Check nonempty; return top value and decrement top pointer.

PEEK

Return top value without removing it; do not change top pointer.

MODEL / FOLLOW THE FLOW

Empty top=-1 → **Push A top=0** → **Push B top=1** → **Pop gives B top=0**

An array-backed example uses top = -1 for an empty stack and zero-based slots.

EXAM-READY STATEMENT

Always name the stack convention (top at last occupied index) and preserve the return order under LIFO.

WORDS THAT EARN MARKS

LIFO Last in first out

Top Last occupied index

Overflow Push onto full

Underflow Pop from empty

1.4.2(b,c) / STEP-BY-STEP WORKED EXAMPLE

Trace operations on a capacity-three stack

Follow each stage in order. Explain both the change and why it happens before you try the exam questions.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
0. Empty	[_, _, _], top=-1	The stack has no items.
1. PUSH A	[A, _, _], top=0	Store in the first slot.
2. PUSH B	[A, B, _], top=1	B is now the top element.
3. PUSH C	[A, B, C], top=2	Stack is full.
4. POP	returns C; top=1	Next POP returns B; pushing D is now possible.

TRANSFER IT TO AN EXAM ANSWER

Always name the stack convention (top at last occupied index) and preserve the return order under LIFO.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check intermediate values and key relationships, not just the final result.

1.4.2(b,c) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Original teacher-written practice questions (not OCR past-paper questions). Work in black pen; use the next page to check.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 5 MARKS

Trace PUSH(4), PUSH(9), POP(), PUSH(7), PEEK(). Give returned values and final stack.

QUESTION 2 | 4 MARKS

Explain both overflow and underflow for an array stack of capacity two.

1.4.2(b,c) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 5 MARKS

- POP returns 9.
- Final stack bottom -> top: [4,7].
- PEEK returns 7 and does not remove it.

Q2 / 4 MARKS

- Overflow occurs when PUSH is attempted on a full stack.
- Underflow occurs when POP is attempted on an empty stack.
- Bounds checks prevent invalid array access.

COMMON MISCONCEPTION

PEEK does not remove data. POP changes the size even if the physical memory slot has not been cleared.

Green pen: List returned values on a separate line from the stored stack.

1.4.2(b,c) / BUILD THE MENTAL MODEL

First in, first out uses front and rear

An array queue can reuse freed slots by wrapping indices modulo capacity. Track count to distinguish full from empty.

THREE PIECES OF THE IDEA

ENQUEUE

Add at the rear after checking capacity.

DEQUEUE

Remove from the front after checking nonempty.

CIRCULAR

Update indices modulo array capacity; count disambiguates full versus empty.

MODEL / FOLLOW THE FLOW

front = 0 → **Enqueue A,B,C** → **Dequeue A** → **Enqueue D at wrap**

In this convention front is the index of the next item to remove; rear is the index of the next free slot.

EXAM-READY STATEMENT

Do not treat front and rear as universally defined; state whether rear means final occupied or next free index.

WORDS THAT EARN MARKS

FIFO	First in first out	**Front**	Next removed
Rear	Next inserted	**MOD**	Wrap index

1.4.2(b,c) / STEP-BY-STEP WORKED EXAMPLE

Circular queue of capacity three

Follow each stage in order. Explain both the change and why it happens before you try the exam questions.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
0. Empty	front=0 rear=0 count=0	Use rear as the next insertion index.
1. Enqueue A	slot 0=A; rear=1 count=1	Front remains 0.
2. Enqueue B,C	slots 1=B,2=C; rear=0 count=3	Queue full; rear wraps (2+1) MOD 3.
3. Dequeue	returns A; front=1 count=2	The freed slot is zero.
4. Enqueue D	slot 0=D; rear=1 count=3	Logical order is B,C,D although physical slots are D,B,C.

TRANSFER IT TO AN EXAM ANSWER

Do not treat front and rear as universally defined; state whether rear means final occupied or next free index.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check intermediate values and key relationships, not just the final result.

1.4.2(b,c) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Original teacher-written practice questions (not OCR past-paper questions). Work in black pen; use the next page to check.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 5 MARKS

Starting empty with capacity three, enqueue 1,2,3; dequeue once; enqueue 4. State physical slots and logical order.

QUESTION 2 | 4 MARKS

Explain why front = rear alone cannot distinguish a circular queue's full and empty conditions.

1.4.2(b,c) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 5 MARKS

- Slots become [4,2,3] using next-free-rear convention.
- Logical FIFO order is 2,3,4.
- front=1, rear=1, count=3.

Q2 / 4 MARKS

- Indices can be equal after all items are removed or after all slots are filled.
- Maintain count, a separate full flag, or reserve a spare slot.

COMMON MISCONCEPTION

Circular does not mean items move in memory. Pointers wrap and freed slots are reused.

Green pen: Record both physical array and logical order after each operation.

1.4.2(b,c) / BUILD THE MENTAL MODEL

Change links without shifting every node

A singly linked list stores a value plus a reference to the next node. The head references the first node.

THREE PIECES OF THE IDEA

NODE

A data field plus a next pointer/reference.

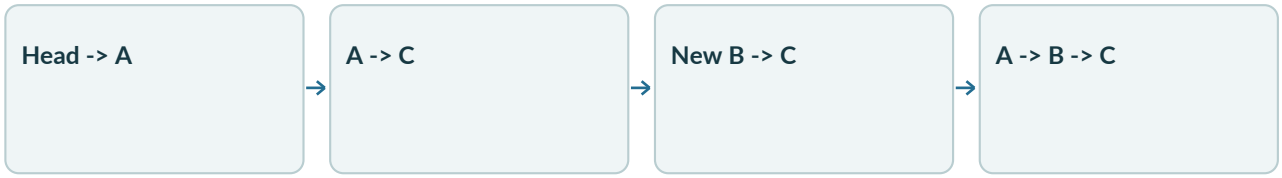
INSERT

Connect new node to successor before reconnecting predecessor or head.

DELETE

Bypass the removed node and update head if required; reclaim storage as appropriate.

MODEL / FOLLOW THE FLOW



To insert between A and C, set B.next to C before setting A.next to B.

EXAM-READY STATEMENT

Pointers change, not the positions of all other stored values. Handle head/tail and empty-list cases explicitly.

WORDS THAT EARN MARKS

Head First-node reference

Null No successor

Pointer Node reference

Traversal Follow links

1.4.2(b,c) / STEP-BY-STEP WORKED EXAMPLE

Insert B into A -> C, then delete A

Follow each stage in order. Explain both the change and why it happens before you try the exam questions.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
1. Before	head -> A -> C -> NULL	A.next refers to C.
2. Make B	B.next = C	Preserve access to C before changing A.
3. Reconnect	A.next = B	Now A -> B -> C.
4. Remove A	head = A.next	New head becomes B.
5. After	head -> B -> C -> NULL	A can be released if not referenced elsewhere.

TRANSFER IT TO AN EXAM ANSWER

Pointers change, not the positions of all other stored values. Handle head/tail and empty-list cases explicitly.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check intermediate values and key relationships, not just the final result.

1.4.2(b,c) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Original teacher-written practice questions (not OCR past-paper questions). Work in black pen; use the next page to check.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 5 MARKS

Describe insertion of new node X immediately after node P in a singly linked list. Include statement order.

QUESTION 2 | 5 MARKS

Explain how to remove the first node from a singly linked list and identify two edge cases.

1.4.2(b,c) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 5 MARKS

- Set X.next = P.next.
- Set P.next = X.
- Maintain any tail pointer if P was originally the tail.

Q2 / 5 MARKS

- Check whether list is empty before deletion.
- Save old head if required, set head = head.next.
- If list becomes empty update tail; reclaim removed node according to implementation.

COMMON MISCONCEPTION

Reversing the assignment order during insertion can make the remainder of the list unreachable.

Green pen: Sketch head and next arrows before and after the operation.

1.4.2(b,c) / BUILD THE MENTAL MODEL

A search tree enforces an ordering invariant

A tree has a root and parent-child edges without cycles. In a binary search tree (BST), left subtree keys are smaller and right subtree keys larger under the chosen duplicate policy.

THREE PIECES OF THE IDEA

ROOT / LEAF

Root has no parent. A leaf has no children.

BINARY TREE

Each node has no more than two children, but the keys need not be sorted.

BST

Left keys < parent < right keys if duplicate keys are disallowed.

MODEL / FOLLOW THE FLOW

Insert 8 → **Insert 3 left** → **Insert 11 right** → **Search 11 right**

A balanced BST can be efficient, but a badly ordered insertion sequence can produce a chain with linear search time.

EXAM-READY STATEMENT

BST ordering is recursive: every key in the whole left subtree must be smaller, not only the immediate left child.

WORDS THAT EARN MARKS

- **Root** Top node
- **Leaf** No children
- **Height** Longest path
- **BST** Ordered keys

1.4.2(b,c) / STEP-BY-STEP WORKED EXAMPLE

Build and search for a key

Follow each stage in order. Explain both the change and why it happens before you try the exam questions.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
1. Insert 8	root=8	First item becomes root.
2. Insert 3	3 < 8 -> left child	Compare at every visited node.
3. Insert 11	11 > 8 -> right child	11 becomes right child.
4. Insert 5	5 < 8; 5 > 3	5 is right child of node 3.
5. Search 5	8 -> 3 -> 5	At each step choose left or right by comparison.

TRANSFER IT TO AN EXAM ANSWER

BST ordering is recursive: every key in the whole left subtree must be smaller, not only the immediate left child.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check intermediate values and key relationships, not just the final result.

1.4.2(b,c) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Original teacher-written practice questions (not OCR past-paper questions). Work in black pen; use the next page to check.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 5 MARKS

Insert 10, 6, 14, 8, 4 into an empty BST. Describe the edges and the search path to 8.

QUESTION 2 | 4 MARKS

Explain why inserting sorted values into an ordinary unbalanced BST may slow searches.

1.4.2(b,c) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 5 MARKS

- Root 10; left 6, right 14; node 6 has left 4 and right 8.
- Search path 10 -> 6 -> 8.

Q2 / 4 MARKS

- Sorted order can make each new node the right or left child of the last.
- Height grows to n rather than approximately log2(n).
- Search can then require O(n) comparisons.

COMMON MISCONCEPTION

A binary tree is not necessarily a BST. Ordering is an additional invariant.

Green pen: Write the comparator direction along each search arrow.

1.4.2(c) / BUILD THE MENTAL MODEL

In-order on a BST yields sorted keys

Traversal order depends on whether a node is processed before, between or after its subtrees.

THREE PIECES OF THE IDEA

PRE-ORDER

Node, left subtree, right subtree.

IN-ORDER

Left subtree, node, right subtree; sorted keys for a BST.

POST-ORDER

Left subtree, right subtree, node.

MODEL / FOLLOW THE FLOW

Root 8 → **Left 3 with right 5** → **Right 11** → **Visit in order**

For BST with root 8, left 3 (right child 5) and right 11: in-order outputs 3,5,8,11.

EXAM-READY STATEMENT

When deleting a BST node with two children, replace it consistently by an in-order predecessor or successor, then remove the moved node.

WORDS THAT EARN MARKS

Traversal Visit each node

In-order Left-node-right

Successor Next larger key

Deletion Preserve ordering

1.4.2(c) / STEP-BY-STEP WORKED EXAMPLE

Traverse and delete node 8 from a BST

Follow each stage in order. Explain both the change and why it happens before you try the exam questions.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
1. Tree	Root 8; left 3 (right 5); right 11	All keys follow BST order.
2. In-order	3, 5, 8, 11	Visit left, node, right.
3. Pre-order	8, 3, 5, 11	Visit node before subtrees.
4. Post-order	5, 3, 11, 8	Visit node after subtrees.
5. Delete root	Replace 8 with in-order successor 11	11 has no left subtree here; reconnect remaining nodes.

TRANSFER IT TO AN EXAM ANSWER

When deleting a BST node with two children, replace it consistently by an in-order predecessor or successor, then remove the moved node.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check intermediate values and key relationships, not just the final result.

1.4.2(c) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Original teacher-written practice questions (not OCR past-paper questions). Work in black pen; use the next page to check.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 5 MARKS

For BST root 6, children 2 and 9, with 2 having right child 4, give in-, pre- and post-order traversals.

QUESTION 2 | 4 MARKS

Outline deletion of a BST node that has two children without breaking the ordering property.

1.4.2(c) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 5 MARKS

- In-order: 2,4,6,9.
- Pre-order: 6,2,4,9.
- Post-order: 4,2,9,6.

Q2 / 4 MARKS

- Find in-order successor (minimum of right subtree) or predecessor (maximum left).
- Replace target key/value with that key/value.
- Delete the successor/predecessor from its original location and reconnect its child if needed.

COMMON MISCONCEPTION

Pre-order and post-order results are not generally sorted even for a BST.

Green pen: Write the left-node-right pattern above each traversal before tracing.

1.4.2(b,c) / BUILD THE MENTAL MODEL

An edge may have a direction and a weight

A graph contains vertices and edges. Choose a representation suited to whether the graph is dense or sparse and whether direction matters.

THREE PIECES OF THE IDEA

UNDIRECTED

An edge A-B permits travel in either direction.

DIRECTED

A -> B is an ordered relationship; reverse edge is separate.

REPRESENTATION

Adjacency lists store neighbours; adjacency matrices use a cell for each ordered pair.

MODEL / FOLLOW THE FLOW

Vertices A,B,C → **Edges A->B, B->C** → **Adjacency list** → **Visit each once**

Directed list: A:[B], B:[C], C:[]; its adjacency matrix is asymmetric because edges are one-way.

EXAM-READY STATEMENT

Search orders depend on specified adjacency order. State that convention before writing a single valid traversal.

WORDS THAT EARN MARKS

- **Vertex** Graph node
- **Edge** Connection
- **Directed** One-way relation
- **Adjacency** Neighbours

1.4.2(b,c) / STEP-BY-STEP WORKED EXAMPLE

Trace BFS and DFS on a small graph

Follow each stage in order. Explain both the change and why it happens before you try the exam questions.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
1. Graph	A-B, A-C, B-D, C-E	Edges are undirected; neighbours processed alphabetically.
2. BFS start	Queue [A], visited {A}	Dequeue A; enqueue B and C.
3. BFS order	A, B, C, D, E	Breadth-first explores each level with a queue.
4. DFS start	A -> B -> D, backtrack	Depth-first follows an unvisited path.
5. DFS order	A, B, D, C, E	Assumes neighbours visited alphabetically and marked on discovery.

TRANSFER IT TO AN EXAM ANSWER

Search orders depend on specified adjacency order. State that convention before writing a single valid traversal.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check intermediate values and key relationships, not just the final result.

1.4.2(b,c) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Original teacher-written practice questions (not OCR past-paper questions). Work in black pen; use the next page to check.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 5 MARKS

For undirected edges A-B, A-C, B-D, C-E, give BFS and DFS orders from A using alphabetical neighbours.

QUESTION 2 | 4 MARKS

Compare adjacency lists with adjacency matrices for a sparse directed network.

1.4.2(b,c) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 5 MARKS

- BFS: A,B,C,D,E.
- DFS: A,B,D,C,E with the specified neighbour ordering.
- BFS uses a queue; DFS uses recursion or a stack.

Q2 / 4 MARKS

- Lists store present outgoing edges and can be space-efficient when sparse.
- Matrix uses V x V positions and gives constant-time direct edge lookup.
- Direction means matrix A,B and B,A may differ.

COMMON MISCONCEPTION

BFS and DFS are not one universal ordering unless the neighbour selection rule is provided.

Green pen: Mark vertices as visited to avoid infinite loops in cyclic graphs.

1.4.2(b,c) / BUILD THE MENTAL MODEL

A good hash chooses a slot; collisions need a policy

A hash table converts a key to an array index. Two distinct keys can map to the same slot; the table must handle that collision.

THREE PIECES OF THE IDEA

HASH FUNCTION

Computes an index, e.g. key MOD capacity for simple integer examples.

COLLISION

Two keys receive the same initial index.

RESOLUTION

Chaining stores multiple items per bucket; open addressing probes another slot.

MODEL / FOLLOW THE FLOW

Key 17 -> 3 → **Key 24 -> 3** → **Probe next free slot** → **Search same sequence**

With seven slots, 17 MOD 7 = 3 and 24 MOD 7 = 3; collision handled using linear probing to slot 4.

EXAM-READY STATEMENT

Deletion in open addressing must preserve the probe chain, commonly with a deleted marker or careful rehashing.

WORDS THAT EARN MARKS

- **Hash** Key-to-index
- **Bucket** Slot or chain
- **Probe** Alternative slot
- **Tombstone** Deleted marker

1.4.2(b,c) / STEP-BY-STEP WORKED EXAMPLE

Insert/search/delete in capacity-7 open-address table

Follow each stage in order. Explain both the change and why it happens before you try the exam questions.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
1. h(k)	k MOD 7	Slots indexed 0 to 6.
2. Insert 10	10 MOD 7 = 3 -> slot 3	Slot was empty.
3. Insert 17	17 MOD 7 = 3, probe slot 4	Collision; store at first free slot.
4. Search 17	Check slot 3 then slot 4	Use identical probe sequence.
5. Delete 10	Mark slot 3 DELETED, not EMPTY	Otherwise a later search for 17 could stop too early.

TRANSFER IT TO AN EXAM ANSWER

Deletion in open addressing must preserve the probe chain, commonly with a deleted marker or careful rehashing.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check intermediate values and key relationships, not just the final result.

1.4.2(b,c) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Original teacher-written practice questions (not OCR past-paper questions). Work in black pen; use the next page to check.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 5 MARKS

With hash key MOD 5 and linear probing, insert 7,12,17 into an empty table. Give the slots used.

QUESTION 2 | 4 MARKS

Explain why replacing a deleted collided entry with an EMPTY marker can cause failed lookups.

1.4.2(b,c) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 5 MARKS

- 7 MOD 5 = 2 -> slot 2.
- 12 MOD 5 = 2 -> probe slot 3.
- 17 MOD 5 = 2 -> probe slots 3 then 4.

Q2 / 4 MARKS

- A search stops on an actually empty slot.
- Later colliding records may be beyond the deleted index.
- Mark DELETED (tombstone) or correctly rehash to preserve probe sequence.

COMMON MISCONCEPTION

Hashing is not encryption, and a hash table is not necessarily sorted.

Green pen: Write the table index under each attempt and keep the search probe order identical to insertion.

1.4.3(a,d) / BUILD THE MENTAL MODEL

Turn a written rule into a testable expression

Boolean expressions encode conditions using true/false inputs. A truth table checks the output for every combination.

THREE PIECES OF THE IDEA

AND

All specified inputs must be true.

OR

At least one input must be true (inclusive OR).

NOT

Reverse a truth value; combine brackets to control the scope.

MODEL / FOLLOW THE FLOW

Door rule → **A = staff** → **B = valid code** → **F = A AND B**

A door opens only if a staff credential and valid code are both present: F = A AND B.

EXAM-READY STATEMENT

A truth table with n Boolean inputs has 2^n rows, and inclusive OR is true if either or both operands are true.

WORDS THAT EARN MARKS

- **Input** Condition
- **Output** Decision
- **Truth table** All combinations
- **Expression** Logic rule

APPENDIX EXAMPLES / OCR Boolean symbols >

1.4.3(a,d) / STEP-BY-STEP WORKED EXAMPLE

Build a three-input access rule

Follow each stage in order. Explain both the change and why it happens before you try the exam questions.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
1. Define	A=staff, B=code, C=override	Name every variable and whether TRUE means present.
2. Write	F=(A AND B) OR C	Override opens independently; otherwise both conditions needed.
3. Example	A=1 B=0 C=0 -> F=0	Staff without code and no override: locked.
4. Example	A=0 B=0 C=1 -> F=1	Override alone grants access.
5. Check	Three inputs require 2^3=8 rows	Do not omit input combinations.

TRANSFER IT TO AN EXAM ANSWER

A truth table with n Boolean inputs has 2^n rows, and inclusive OR is true if either or both operands are true.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check intermediate values and key relationships, not just the final result.

1.4.3(a,d) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Original teacher-written practice questions (not OCR past-paper questions). Work in black pen; use the next page to check.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 5 MARKS

A light comes on if motion M is detected AND it is dark D, OR the manual switch S is on. Write and explain the Boolean expression.

QUESTION 2 | 4 MARKS

For F=(A AND B) OR C, calculate outputs for 000, 101, 110 and 111 (A,B,C order).

1.4.3(a,d) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 5 MARKS

- F=(M AND D) OR S.
- Motion and darkness must both be true for automatic activation.
- S=TRUE switches it on independently.

Q2 / 4 MARKS

- 000 -> 0.
- 101 -> 1 (C=1).
- 110 -> 1 (A AND B=1).
- 111 -> 1.

COMMON MISCONCEPTION

OR is inclusive, not exclusive, unless XOR is explicitly specified.

Green pen: State what each variable represents before drawing a table.

1.4.3(d) / BUILD THE MENTAL MODEL

Read the gate at every stage

Logic gates implement Boolean operations; intermediate wire values make a multi-gate circuit easier to follow.

THREE PIECES OF THE IDEA

BASIC

AND, OR and NOT form building blocks; NAND and NOR negate AND/OR outputs.

XOR

TRUE when the two inputs differ. XNOR is TRUE when they match.

CIRCUIT

Mark intermediate outputs so the final truth table follows the circuit, not a guess.

MODEL / FOLLOW THE FLOW

Input A,B → **AND gate X** → **NOT gate Y** → **OR with C -> F**

For the example F=NOT(A AND B) OR C, the intermediate output is X=A AND B, then Y=NOT X.

EXAM-READY STATEMENT

For multi-stage circuits, identify intermediate wires and show their values. NAND = NOT(AND), NOR = NOT(OR).

WORDS THAT EARN MARKS

- **NAND** NOT AND
- **NOR** NOT OR
- **XOR** Different bits
- **XNOR** Same bits

1.4.3(d) / STEP-BY-STEP WORKED EXAMPLE

Trace F=NOT(A AND B) OR C

Follow each stage in order. Explain both the change and why it happens before you try the exam questions.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
1. Inputs	A=1, B=1, C=0	Three specified Boolean inputs.
2. AND	X=1 AND 1 = 1	Output X of first gate.
3. NOT	Y=NOT 1 = 0	Invert X.
4. OR	F=0 OR 0 = 0	Evaluate final stage.
5. Alternative	A=1 B=0 C=0 -> F=1	AND becomes 0; NOT becomes 1.

TRANSFER IT TO AN EXAM ANSWER

For multi-stage circuits, identify intermediate wires and show their values. NAND = NOT(AND), NOR = NOT(OR).

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check intermediate values and key relationships, not just the final result.

1.4.3(d) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Original teacher-written practice questions (not OCR past-paper questions). Work in black pen; use the next page to check.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 4 MARKS

Evaluate F=(A XOR B) AND (NOT C) for A=1, B=0, C=0, showing intermediate results.

QUESTION 2 | 5 MARKS

Describe the truth table for a two-input NAND gate and explain why its output is false only once.

1.4.3(d) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 4 MARKS

- A XOR B = 1 because inputs differ.
- NOT C = 1.
- F=1 AND 1 = 1.

Q2 / 5 MARKS

- 00 -> 1, 01 -> 1, 10 -> 1, 11 -> 0.
- AND is TRUE only when both inputs are TRUE.
- NAND negates the AND output, so only 11 gives FALSE.

COMMON MISCONCEPTION

An XOR gate is not the same as an inclusive OR gate when both inputs are 1.

Green pen: Give the intermediate values in the same order as the circuit.

1.4.3(c) / BUILD THE MENTAL MODEL

Simplify by stating the law used

Boolean algebra has identities that preserve output across all input combinations. De Morgan swaps AND/OR when negating a bracket.

THREE PIECES OF THE IDEA

DE MORGAN

NOT(A AND B) = (NOT A) OR (NOT B); NOT(A OR B) = (NOT A) AND (NOT B).

DOUBLE NEGATION

NOT(NOT A) = A.

REARRANGEMENT

Commutative, associative and distributive laws permit valid reorderings and expansions.

MODEL / FOLLOW THE FLOW

Start NOT(A AND B) → **De Morgan** → **(NOT A) OR (NOT B)** → **Verify by table**

Negation applies to each input AND changes the operator between them.

EXAM-READY STATEMENT

Name each law. NOT(A AND B) becomes NOT A OR NOT B, not NOT A AND NOT B.

WORDS THAT EARN MARKS

- **De Morgan** Negate + swap
- **Distribute** Factor or expand
- **Associate** Regroup
- **Commute** Reorder

1.4.3(c) / STEP-BY-STEP WORKED EXAMPLE

Simplify (A AND B) OR (A AND NOT B)

Follow each stage in order. Explain both the change and why it happens before you try the exam questions.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
1. Start	(A AND B) OR (A AND NOT B)	Identify common A.
2. Distribute	A AND (B OR NOT B)	Factor A using distributive law.
3. Complement	B OR NOT B = 1	Excluded-middle / complement law.
4. Identity	A AND 1 = A	One AND A is A.
5. Check	B=0 or B=1 gives result A	Verify using a two-input truth table.

TRANSFER IT TO AN EXAM ANSWER

Name each law. NOT(A AND B) becomes NOT A OR NOT B, not NOT A AND NOT B.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check intermediate values and key relationships, not just the final result.

1.4.3(c) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Original teacher-written practice questions (not OCR past-paper questions). Work in black pen; use the next page to check.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 5 MARKS

Simplify (P AND Q) OR (P AND NOT Q), naming each law used.

QUESTION 2 | 5 MARKS

Use De Morgan to transform NOT(X OR (Y AND Z)); show the negations and operators.

1.4.3(c) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 5 MARKS

- Factor P: P AND (Q OR NOT Q).
- Complement law: Q OR NOT Q = 1.
- Identity law: P AND 1 = P.

Q2 / 5 MARKS

- NOT X AND NOT(Y AND Z).
- Apply De Morgan within bracket: NOT X AND (NOT Y OR NOT Z).
- Both operator changes must be shown.

COMMON MISCONCEPTION

De Morgan changes the operator as well as distributing NOT.

Green pen: Underline the bracket that is being negated at each step.

1.4.3(b) / BUILD THE MENTAL MODEL

Adjacent ones let a changing variable disappear

A Karnaugh map (K-map) places truth-table outputs in Gray-code order. Group adjacent ones in rectangles of sizes 1, 2, 4 or 8.

THREE PIECES OF THE IDEA

ORDER

For two-bit axis labels use 00, 01, 11, 10.

GROUP

Choose largest available power-of-two groups, allowing overlap and wrap-around.

EXPRESSION

A variable disappearing within a group is omitted from that group term.

WORKED VISUAL / NOT TO SCALE

A / BC	00	01	11	10
0	0	0	1	1
1	0	1	1	0

TWO PAIRS

Group row A=0: m2,m3. Group row A=1: m5,m7.

F = (NOT A AND B) OR (A AND C)

EXAM-READY STATEMENT

If a variable takes both 0 and 1 inside a group, it cancels from the product term.

WORDS THAT EARN MARKS

- **Minterm** Input row with 1
- **Gray code** One bit changes
- **Group** Power of two
- **Implicant** Simplified product

1.4.3(b) / STEP-BY-STEP WORKED EXAMPLE

Simplify F=1 for minterms 2, 3, 5 and 7

Follow each stage in order. Explain both the change and why it happens before you try the exam questions.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
1. Labels	A rows 0,1; BC columns 00,01,11,10	Each adjacent cell differs in one input.
2. Put ones	m2,m3,m5,m7 are 1	All other input rows have F=0.
3. First group	m2 and m3	A=0 and B=1 constant; C varies.
4. Second group	m5 and m7	A=1 and C=1 constant; B varies.
5. Result	F=(NOT A AND B) OR (A AND C)	Combine group products by OR.

TRANSFER IT TO AN EXAM ANSWER

If a variable takes both 0 and 1 inside a group, it cancels from the product term.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check intermediate values and key relationships, not just the final result.

1.4.3(b) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Original teacher-written practice questions (not OCR past-paper questions). Work in black pen; use the next page to check.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 5 MARKS

For three inputs A,B,C, F=1 on minterms 0,1,2,3. Give a simplified expression and explain the group.

QUESTION 2 | 5 MARKS

Explain why K-map columns are not in ordinary binary order 00,01,10,11.

1.4.3(b) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 5 MARKS

- All four rows have A=0 while B and C vary.
- One four-cell group gives F=NOT A.

Q2 / 5 MARKS

- Gray-code order 00,01,11,10 keeps adjacent labels different by one bit.
- This allows neighbouring ones to differ in only one variable, which can then be removed.

COMMON MISCONCEPTION

A group of three cells is invalid; group sizes must be powers of two.

Green pen: Highlight the variables constant within the group, not those that change.

1.4.3(b) / BUILD THE MENTAL MODEL

Wrap-around groups include all four corners

In a 4-variable map, each axis is in Gray-code order. Opposite edges touch; the four corner cells can form one 4-cell group.

THREE PIECES OF THE IDEA

FOUR AXES BITS

Use AB across columns and CD down rows, both ordered 00,01,11,10.

WRAP

First and last rows touch; first and last columns touch.

OVERLAP

One cell may belong to multiple groups if this allows fewer/larger terms.

WORKED VISUAL / NOT TO SCALE

CD / AB	00	01	11	10
00	1	0	0	1
01	0	0	0	0
11	0	0	0	0
10	1	0	0	1

FOUR CORNERS

One group wraps across **both** pairs of edges.

B = 0 and D = 0 remain fixed.

F = NOT B AND NOT D

EXAM-READY STATEMENT

Always consider corner and opposite-edge adjacency before settling for multiple small groups.

WORDS THAT EARN MARKS

- **K-map** Gray-code grid
- **Wrap** Opposite edge
- **Constant** Keep variable
- **Overlap** Reuse a 1

1.4.3(b) / STEP-BY-STEP WORKED EXAMPLE

Simplify the four-corner pattern

Follow each stage in order. Explain both the change and why it happens before you try the exam questions.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
1. Grid	AB: 00 01 11 10; CD same order	Use Gray-coded axes.
2. Ones	(AB,CD): (00,00),(10,00),(00,10),(10,10)	These are the four corners.
3. Group	One 4-cell rectangle wraps both edges	All four positions are adjacent through wrapping.
4. Constants	B=0 and D=0	A and C vary, so they disappear.
5. Result	F=(NOT B) AND (NOT D)	Equivalent to NOT(B OR D).

TRANSFER IT TO AN EXAM ANSWER

Always consider corner and opposite-edge adjacency before settling for multiple small groups.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check intermediate values and key relationships, not just the final result.

1.4.3(b) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Original teacher-written practice questions (not OCR past-paper questions). Work in black pen; use the next page to check.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 5 MARKS

A 4-variable K-map has ones only in the four corners using AB columns and CD rows. Give the simplest expression and justify it.

QUESTION 2 | 5 MARKS

A second group has all cells with A=1 and C=1. Give its term and combine it by OR with the four-corner expression.

1.4.3(b) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 5 MARKS

- Four corners form one 4-cell wrap-around group.
- B=0 and D=0 are constant.
- F=NOT B AND NOT D.

Q2 / 5 MARKS

- All A=1 and C=1 cells give A AND C.
- Combined F=(NOT B AND NOT D) OR (A AND C).
- Overlapping cells may be included in more than one group.

COMMON MISCONCEPTION

Grid edges are not walls. Do not create four singleton groups when one corner group exists.

Green pen: Write down the axis order and mark wrap connections on both pairs of edges.

1.4.3(e) / BUILD THE MENTAL MODEL

Add two input bits without a carry-in

A half adder adds two one-bit inputs. It produces a one-bit sum and a carry-out.

THREE PIECES OF THE IDEA

SUM

S = A XOR B; the sum is one when inputs differ.

CARRY

C = A AND B; carry is one only for 1+1.

LIMITATION

A half adder has no carry-in from a lower-order bit.

WORKED VISUAL / NOT TO SCALE



Input 1 + 1 yields sum 0 and carry 1, i.e. binary 10.

EXAM-READY STATEMENT

Draw two gates fed by both A and B: XOR to sum, AND to carry. A half adder cannot accept Cin.

WORDS THAT EARN MARKS

- **XOR** Different inputs
- **Carry** Extra binary place
- **Half adder** No Cin
- **Truth table** Four input rows

1.4.3(e) / STEP-BY-STEP WORKED EXAMPLE

Trace all half-adder input combinations

Follow each stage in order. Explain both the change and why it happens before you try the exam questions.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
0+0	A=0 B=0 -> S=0 C=0	No ones at either output.
0+1	A=0 B=1 -> S=1 C=0	Different bits, no carry.
1+0	A=1 B=0 -> S=1 C=0	Same sum as the previous row.
1+1	A=1 B=1 -> S=0 C=1	Binary 10 represents decimal 2.

TRANSFER IT TO AN EXAM ANSWER

Draw two gates fed by both A and B: XOR to sum, AND to carry. A half adder cannot accept Cin.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check intermediate values and key relationships, not just the final result.

1.4.3(e) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Original teacher-written practice questions (not OCR past-paper questions). Work in black pen; use the next page to check.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 4 MARKS

Give the half-adder outputs for A=1, B=1, and name the gates that produce each output.

QUESTION 2 | 4 MARKS

Explain why a half adder alone is insufficient for a middle bit in multi-bit addition.

1.4.3(e) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 4 MARKS

- Sum S=0 using XOR.
- Carry C=1 using AND.
- Output 10 represents decimal 2.

Q2 / 4 MARKS

- A middle column must add A, B and carry-in from the previous column.
- A half adder accepts only A and B; a full adder supports three inputs.

COMMON MISCONCEPTION

The carry output is not the same wire as the sum output.

Green pen: Check the 1+1 row: sum 0 and carry 1.

1.4.3(e) / BUILD THE MENTAL MODEL

A full adder accepts three one-bit inputs

A full adder adds A, B and Cin. It produces S and Cout; two half adders and an OR gate form a standard implementation.

THREE PIECES OF THE IDEA

FIRST HALF ADDER

S1 = A XOR B; C1 = A AND B.

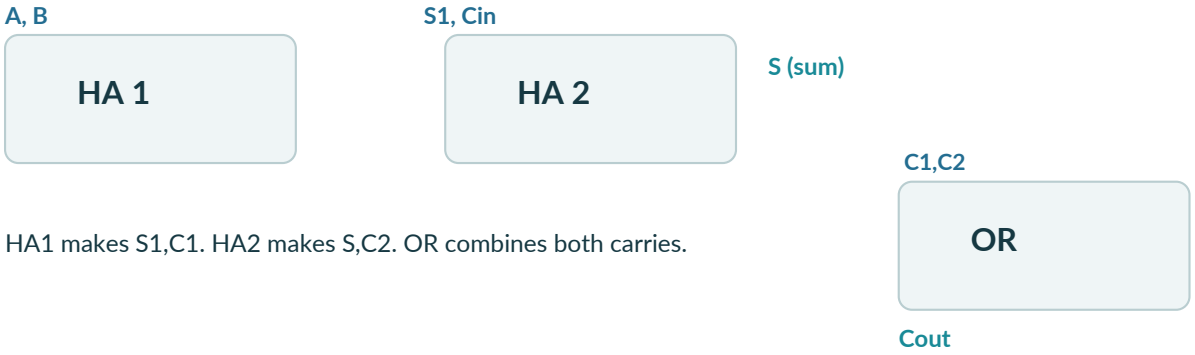
SECOND HALF ADDER

S = S1 XOR Cin; C2 = S1 AND Cin.

COMBINE CARRIES

Cout = C1 OR C2; equivalently AB OR (A XOR B)Cin.

WORKED VISUAL / NOT TO SCALE



HA1 makes S1,C1. HA2 makes S,C2. OR combines both carries.

EXAM-READY STATEMENT

For a full adder, S = A XOR B XOR Cin and Cout = (A AND B) OR (Cin AND (A XOR B)).

WORDS THAT EARN MARKS

Cin	Incoming carry	**Cout**	Outgoing carry
HA	Half adder	**XOR**	Sum parity

1.4.3(e) / STEP-BY-STEP WORKED EXAMPLE

Trace A=1, B=0, Cin=1

Follow each stage in order. Explain both the change and why it happens before you try the exam questions.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
1. First sum	S1 = 1 XOR 0 = 1	First half adder sum.
2. First carry	C1 = 1 AND 0 = 0	First half adder carry.
3. Second sum	S = 1 XOR 1 = 0	Second half adder sum.
4. Second carry	C2 = 1 AND 1 = 1	Second half adder carry.
5. Combine	Cout = 0 OR 1 = 1	Binary output CoutS = 10 (decimal 2).

TRANSFER IT TO AN EXAM ANSWER

For a full adder, S = A XOR B XOR Cin and Cout = (A AND B) OR (Cin AND (A XOR B)).

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check intermediate values and key relationships, not just the final result.

1.4.3(e) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Original teacher-written practice questions (not OCR past-paper questions). Work in black pen; use the next page to check.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 5 MARKS

A full adder receives A=1, B=1, Cin=1. Calculate S and Cout through two half adders.

QUESTION 2 | 5 MARKS

Explain how two half adders plus one OR gate produce a full-adder circuit.

1.4.3(e) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 5 MARKS

- HA1: S1=0, C1=1.
- HA2 with Cin=1: S=1, C2=0.
- Cout=1 OR 0=1, output 11 (decimal 3).

Q2 / 5 MARKS

- First half adder receives A,B -> S1 and C1.
- Second receives S1,Cin -> S and C2.
- OR combines C1,C2 to Cout.

COMMON MISCONCEPTION

Cout is not just A AND B because Cin can produce a carry when one of A/B is 1.

Green pen: Label both intermediate carry outputs before drawing the final OR gate.

1.4.3(e) / BUILD THE MENTAL MODEL

Store the input bit at the active clock edge

A D-type flip-flop stores one bit. In a rising-edge-triggered version, the input D is sampled at the clock's rising edge and output Q holds until the next rising edge.

THREE PIECES OF THE IDEA

DATA

D is the bit offered for storage.

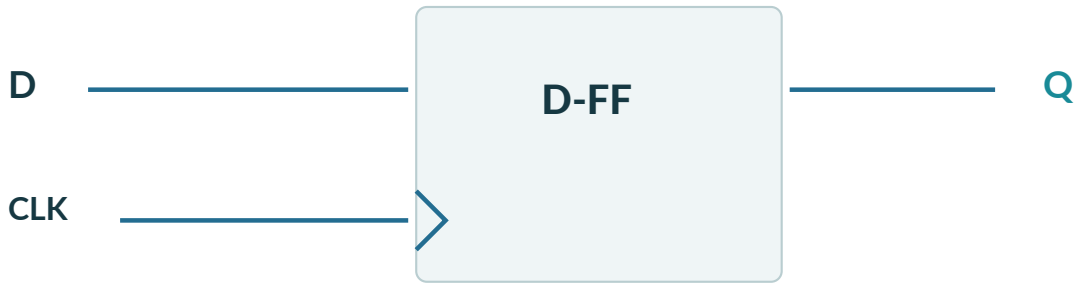
CLOCK

An edge-triggered symbol uses a small triangle at CLK; a rising edge causes sampling.

OUTPUT

Q is the stored bit. The complementary output, if shown, is NOT Q.

WORKED VISUAL / NOT TO SCALE



Rising clock edge: Q copies D. Between edges: Q holds.

EXAM-READY STATEMENT

A D-type flip-flop is storage, not a gate that continuously copies D. A collection of flip-flops can form a register.

WORDS THAT EARN MARKS

D	Data input	**CLK**	Clock input
Q	Stored output	**Edge**	Sampling instant

1.4.3(e) / STEP-BY-STEP WORKED EXAMPLE

Trace an edge-triggered register bit

Follow each stage in order. Explain both the change and why it happens before you try the exam questions.

TRACE / DECISION TABLE

STEP	VALUE OR ACTION	REASON / INTERPRETATION
0. Initial	Q=0	Known stored state.
1. Between edges	D changes to 1; Q stays 0	No active clock edge.
2. Rising edge 1	D=1 -> Q=1	Sample the present input.
3. Between edges	D changes to 0; Q stays 1	Still holding the previous value.
4. Rising edge 2	D=0 -> Q=0	Update on this clock edge.

TRANSFER IT TO AN EXAM ANSWER

A D-type flip-flop is storage, not a gate that continuously copies D. A collection of flip-flops can form a register.

Exam habit: state the condition, describe the mechanism, and link it to the specific outcome. Check intermediate values and key relationships, not just the final result.

1.4.3(e) / INDEPENDENT EXAM PRACTICE

Use the model without copying it

Original teacher-written practice questions (not OCR past-paper questions). Work in black pen; use the next page to check.

BLACK PEN: ANSWER FIRST. THEN CHECK THE SEPARATE MARK POINTS.

QUESTION 1 | 5 MARKS

A rising-edge D-flop starts Q=0. D=1 at edge 1, D=0 at edge 2, and D=1 at edge 3. State Q after each edge.

QUESTION 2 | 4 MARKS

Explain why a D-flop output need not change immediately when its data input changes.

1.4.3(e) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct, explain

Check method and vocabulary, not just the final number. These are teacher-written points, not an OCR mark scheme.

Q1 / 5 MARKS

- After edge 1 Q=1; after edge 2 Q=0; after edge 3 Q=1.
- Each active edge captures D at that moment; between edges Q holds.

Q2 / 4 MARKS

- For an edge-triggered device the state updates only on the configured clock transition.
- Between active edges, stored Q stays constant even if D changes.

COMMON MISCONCEPTION

A D-type flip-flop is not a toggle flip-flop: it copies D on the active edge.

Green pen: Mark the rising clock edges before tracing Q.